DEADLOCK

Deadlock is a state in an operating system where two or more processes are stuck forever because each is waiting for a resource held by another. It happens only when four conditions exist: mutual exclusion, hold and wait, no preemption, and circular wait. For example, P1 holds R1 and needs R2, while P2 holds R2 and needs R1, so both wait forever. Deadlock can be handled using prevention, avoidance (Banker's algorithm), or detection and recovery.

How Does Deadlock Occur in OS?
A process in an operating system typically uses resources in the following sequence:

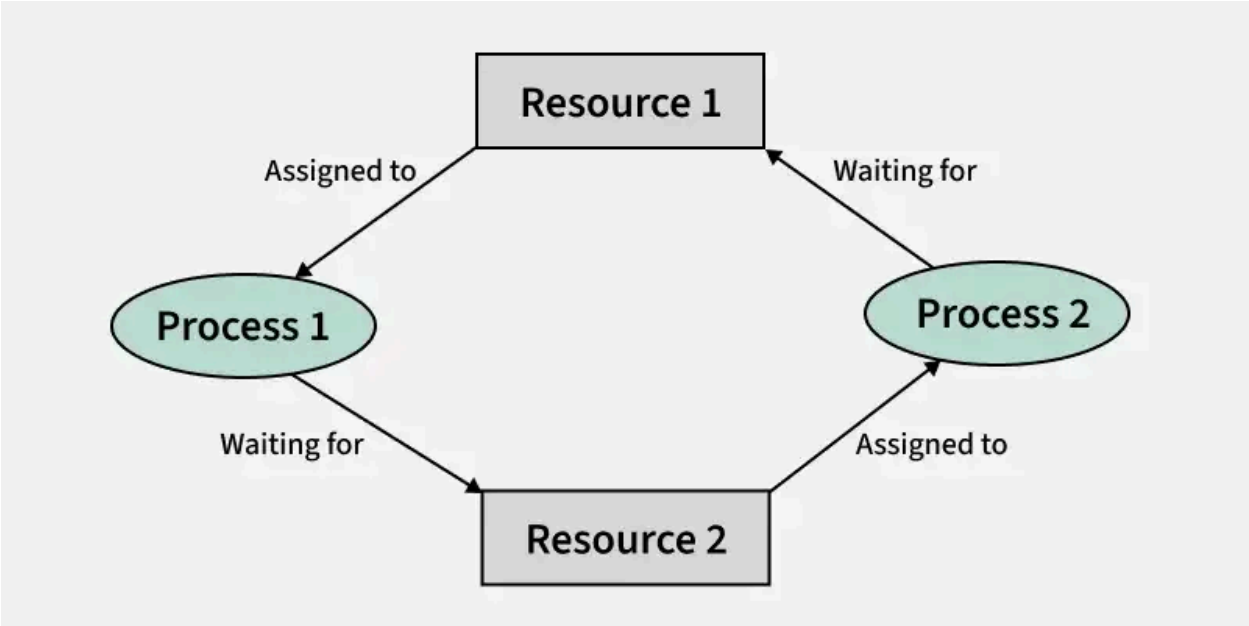
- Request a resource
- Use the resource
- Release the resource

Deadlock arises when processes hold some resources while waiting for others.

Example:

- Process P1 holds Resource R1 and requests R2.
- Process P2 holds Resource R2 and requests R1.

Neither process can proceed causing a deadlock.



Examples of Deadlock
There are several examples of deadlock. Some of them are mentioned below.

1. The system has 2 tape drives. P0 and P1 each hold one tape drive and each needs another one.

2. Semaphores A and B, initialized to 1, P0, and P1 are in deadlock as follows:

- P0 executes wait(A) and preempts.
- P1 executes wait(B).
- Now P0 and P1 enter in deadlock.

P0 Action	P1 Action
wait(A)	wait(B)
wait(B)	wait(A)

3. Assume the space is available for allocation of 200K bytes, and the following sequence of events occurs.

P0	P1
Request 80KB;	Request 70KB;
Request 60KB;	Request 80KB;

Deadlock occurs if both processes progress to their second request.

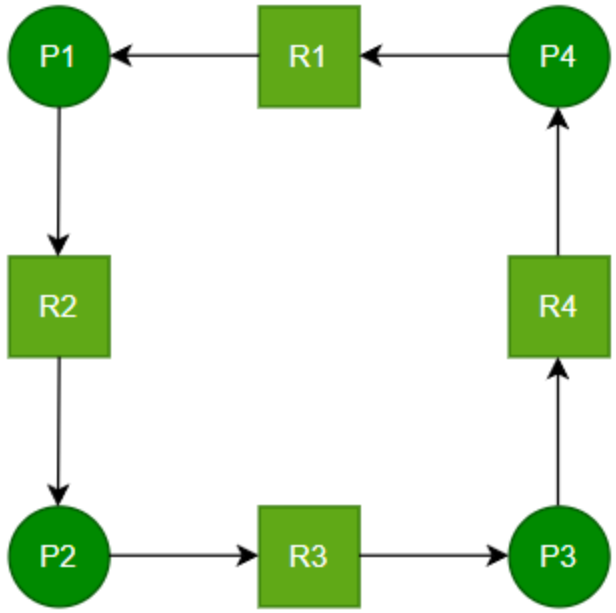
Necessary Conditions for Deadlock in OS
Deadlock can arise if the following four conditions hold simultaneously (Necessary Conditions)

1. **Mutual Exclusion**: Only one process can use a resource at any given time i.e. the resources are non-sharable.

2. **Hold and Wait:** A process is holding at least one resource at a time and is waiting to acquire other resources held by some other process.

3. **No Preemption:** A resource cannot be taken from a process unless the process releases the resource.

4. **Circular Wait:** set of processes are waiting for each other in a circular fashion. For example, imagine four processes P1, P2, P3, and P4 and four resources R1, R2, R3, and R4.



The above image demonstrates a circular wait deadlock, here's how:

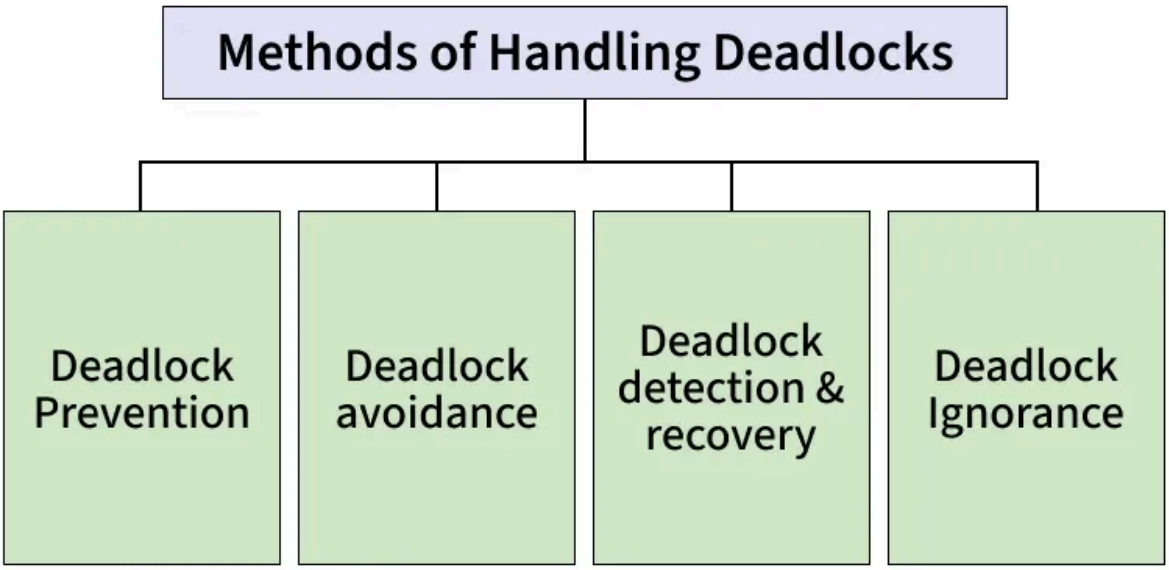
- P1 is holding R1 and waiting for R2 (which is held by P2).
- P2 is holding R2 and waiting for R3 (which is held by P3).
- P3 is holding R3 and waiting for R4 (which is held by P4).
- P4 is holding R4 and waiting for R1 (which is held by P1).

Handling Deadlocks

Deadlock handling methods are strategies used in operating systems to ensure processes do not remain permanently blocked, maintaining smooth execution and system reliability.

Methods of Handling Deadlocks

There are four approaches to dealing with deadlocks.



1. Deadlock Prevention
Deadlock Prevention is a method of handling deadlocks where the operating system ensures that at least one of the necessary conditions for deadlock (mutual exclusion, hold and wait, no preemption, circular wait) never occurs. By breaking these conditions in advance, the system prevents deadlock from happening.

2. Deadlock Avoidance
Deadlock Avoidance is a method where the operating system makes decisions dynamically to ensure the system never enters an unsafe state, thereby avoiding the possibility of deadlock. This is usually done using algorithms like the Banker's Algorithm.

Deadlock avoidance mainly relies on algorithms that check resource allocation before making decisions. The two common types are:

Banker's Algorithm: Used when multiple instances of resources exist. It simulates allocation and ensures that the system remains in a safe state before granting resources.
Resource Allocation Graph (RAG) Algorithm: Used when each resource has only one instance. It checks for cycles in the graph to avoid unsafe states.

3. Deadlock Detection & Recovery

Deadlock Detection: Deadlock detection periodically checks for circular waits and resolves deadlocks by aborting and restarting processes, releasing their resources. It allows unrestricted resource access and immediate request fulfillment, enabling online handling. The main drawback is potential loss due to preemption.

Deadlock Recovery is the process of handling a deadlock after it has occurred. It involves:

1. Process Termination: Aborting one or more deadlocked processes to break the cycle.
2. Resource Preemption: Temporarily taking resources from some processes and reallocating them to others.

4. Deadlock Ignorance

In the Deadlock ignorance method the OS acts like the deadlock never occurs and completely ignores it even if the deadlock occurs. This method only applies if the deadlock occurs very rarely. The algorithm is very simple. It says, " if the deadlock occurs, simply reboot the system and act like the deadlock never occurred." That's why the algorithm is called the Ostrich Algorithm.

Resource Allocation Graph (RAG)

A Resource Allocation Graph (RAG) is a visual way to understand how resources are assigned in an operating system. Instead of using only tables to show which resources are allocated, requested or available, the RAG uses nodes and edges to clearly illustrate relationships between processes and their required resources.

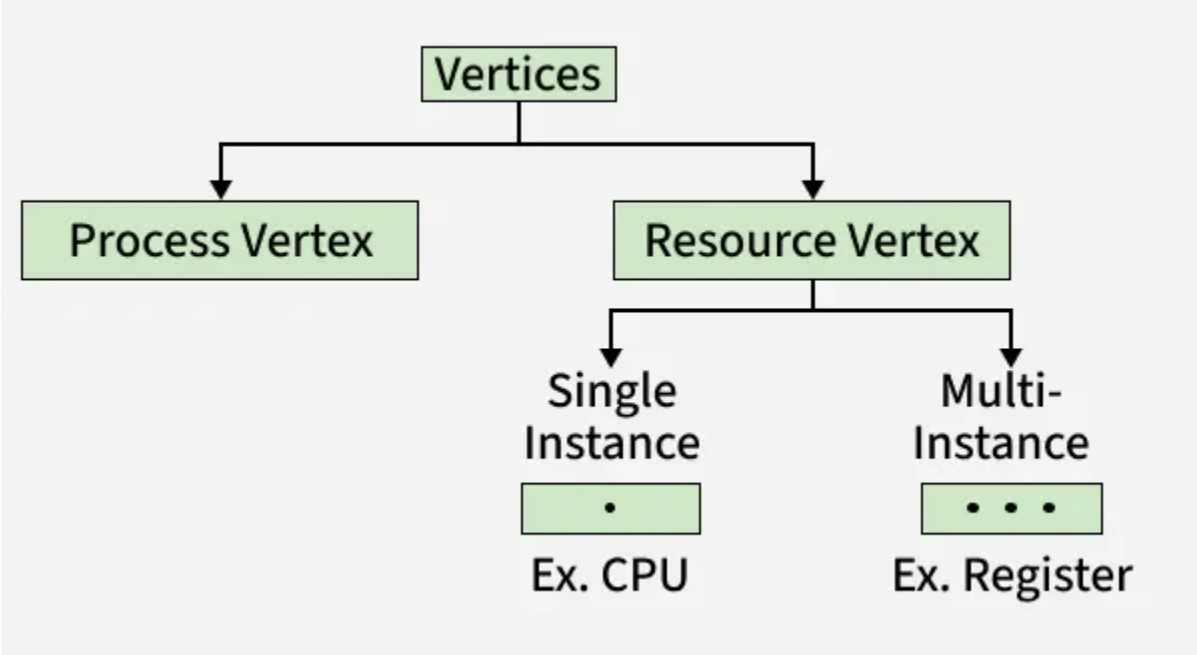
- RAG shows which resources are allocated and which are requested by processes.
- It helps to visualize deadlocks more clearly than tables.
- RAG shows how allocation and requests affect the whole system.
- Processes = circles, Resources = squares and edges show allocation or request.

Types of Vertices in RAG

In a Resource Allocation Graph, there are two types of vertices

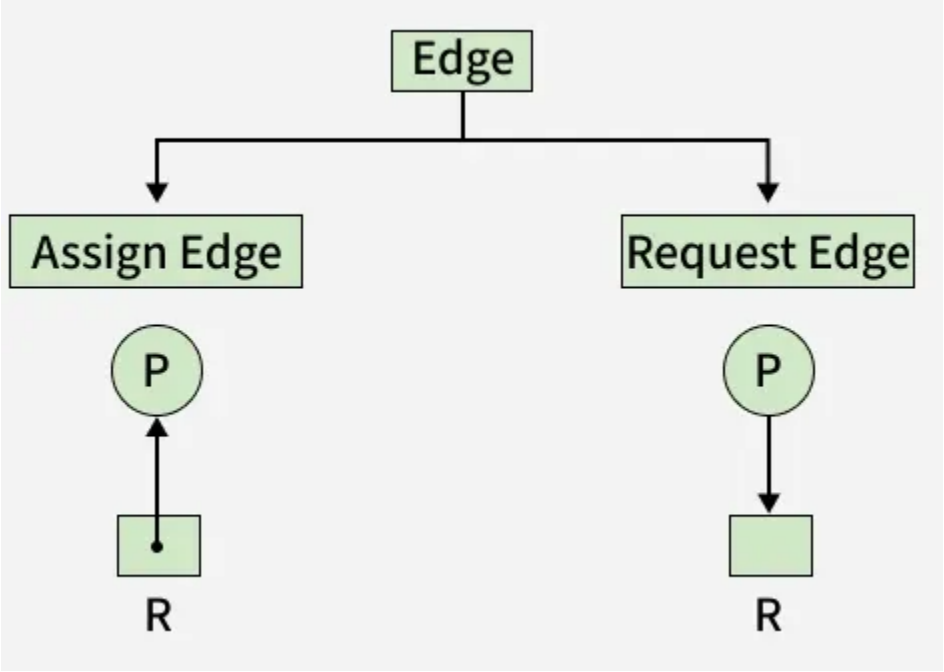
1. **Process Vertex:** Every process will be represented as a process vertex. Generally, the process will be represented with a circle.
2. **Resource Vertex:** Every resource will be represented as a resource vertex. It is also two types:

    1. Single Instance Resource: A resource with only one copy. In RAG, it is shown as a single node and only one process can use it at a time.
    2. Multi Instance Resource: A resource with multiple copies. In RAG, it is shown with multiple instances, so several processes can use different copies simultaneously.



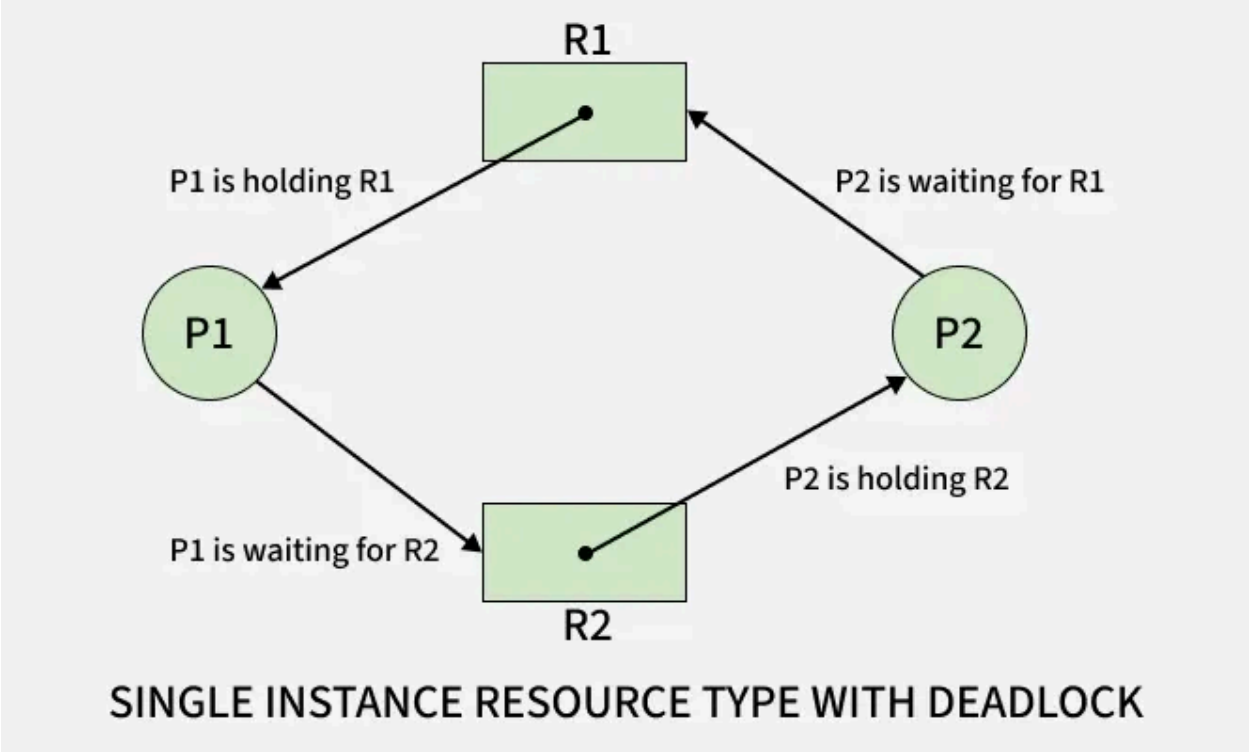
Types of Edges in RAG

1. Assign Edge: If you already assign a resource to a process then it is called Assign edge. This is shown by an arrow from the resource vertex to the process vertex.
2. Request Edge: A request edge represents that a process is currently requesting a resource. This is shown by an arrow from the process vertex to the resource vertex.

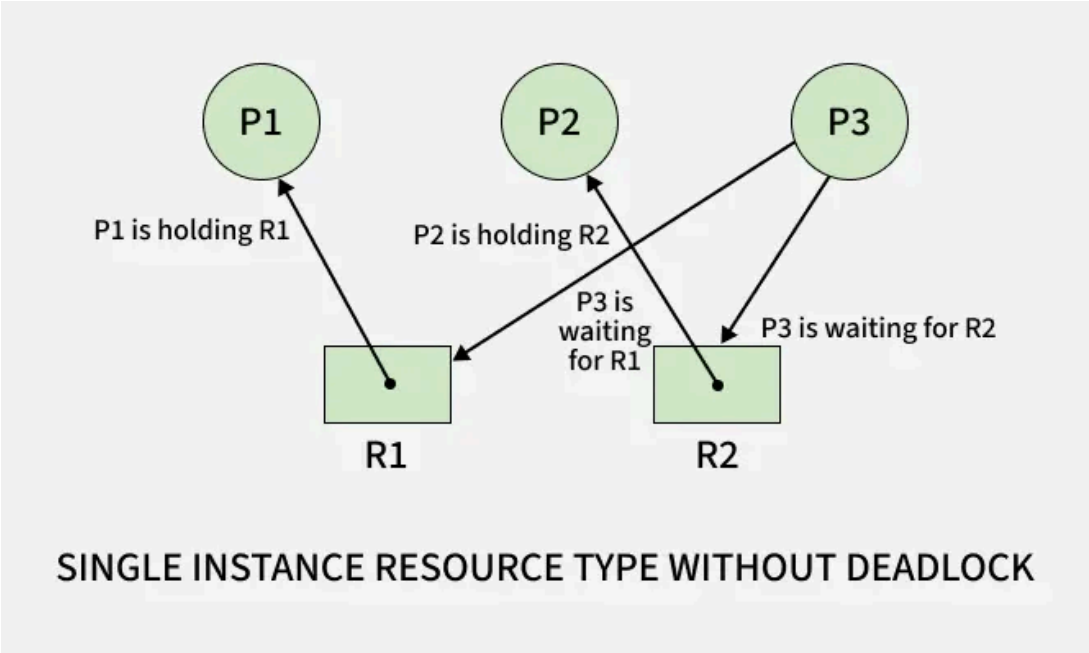


So, if a process is using a resource, an arrow is drawn from the resource node to the process node. If a process is requesting a resource, an arrow is drawn from the process node to the resource node.

Example 1: (Single Instances RAG)

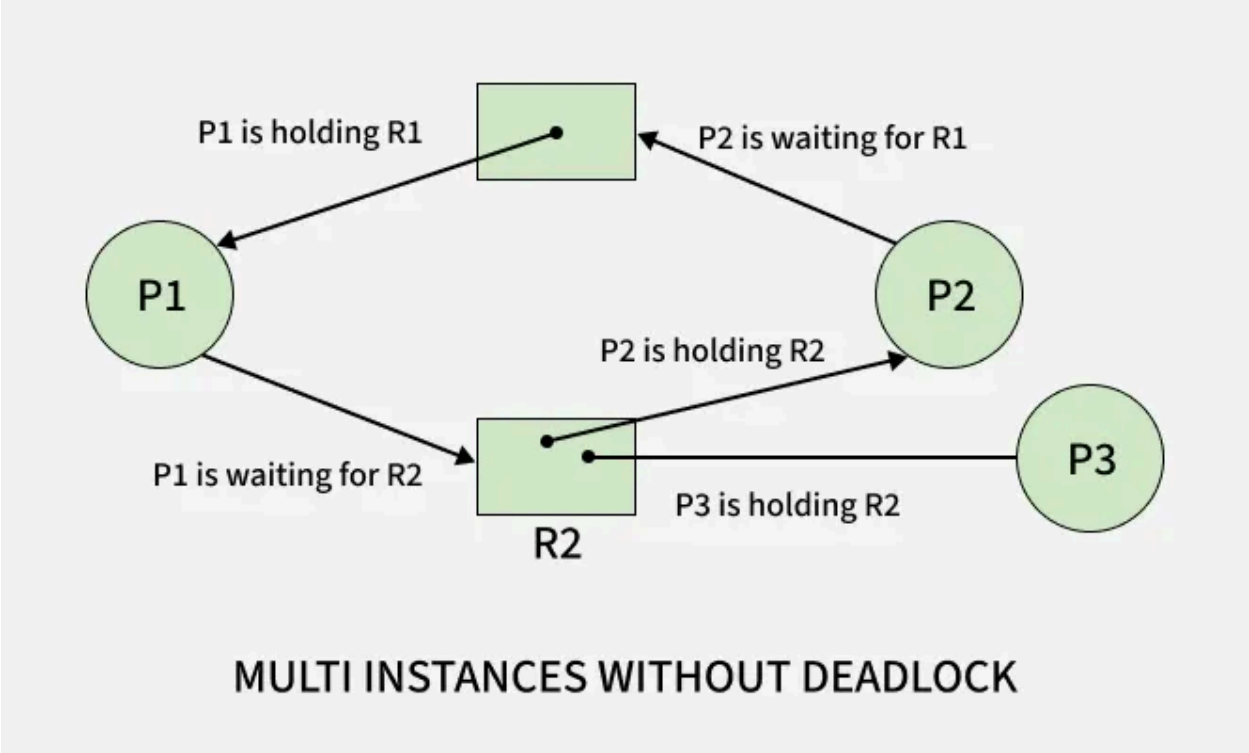


If there is a cycle in the Resource Allocation Graph and each resource in the cycle provides only one instance, then the processes will be in deadlock. For example, if process P1 holds resource R1, process P2 holds resource R2 and process P1 is waiting for R2 and process P2 is waiting for R1, then process P1 and process P2 will be in deadlock.



Here's another example, that shows Processes P1 and P2 acquiring resources R1 and R2 while process P3 is waiting to acquire both resources. In this example, there is no deadlock because there is no circular dependency. So cycle in single-instance resource type is the sufficient condition for deadlock.

Example 2: (Multi-instances RAG)



From the above example, it is not possible to say the RAG is in a safe state or in an unsafe state. So to see the state of this RAG, let's construct the allocation matrix and request matrix.

Process	Allocation		Request	
	Resource		Resource	
	R1	R2	R1	R2
P1	1	0	0	1
P2	0	1	1	0
P3	0	1	0	0

- The total number of processes are three: P1, P2 & P3 and the total number of resources are two: R1 & R2.
- Allocation matrix: For constructing the allocation matrix, just go to the resources and see to which process it is allocated.
  R1 is allocated to P1, therefore write 1 in allocation matrix and similarly, R2 is allocated to P2 as well as P3 and for the remaining element just write 0.
- Request matrix: In order to find out the request matrix, you have to go to the process and see the outgoing edges.
  P1 is requesting resource R2, so write 1 in the matrix and similarly, P2 requesting R1 and for the remaining element write 0.
  So now available resource is = (0, 0).
- Checking deadlock (safe or not)

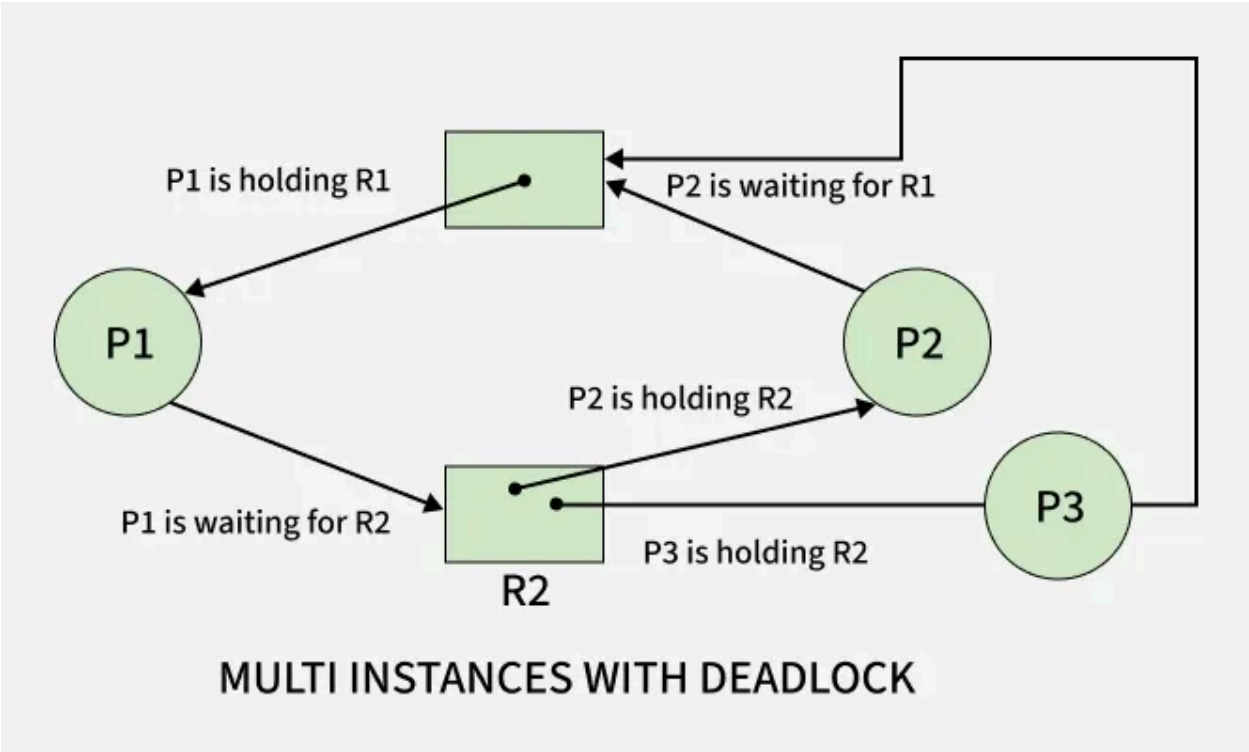
Available = 0 0 (As P3 does not require any extra resource to complete the execution and after
P3 0 1 completion P3 release its own resource)

New Available = 0 1 (As using new available resource we can satisfy the requirment of process P1
P1 1 0 and P1 also release its prevous resource)

New Available = 1 1 (Now easily we can satisfy the requirement of process P2)
P2 0 1

New Available = 1 2

So there is no deadlock in this RAG. Even though there is a cycle, still there is no deadlock. Therefore in multi-instance resource cycle is not sufficient condition for deadlock.



Above example is the same as the previous example except that, the process P3 requesting for resource R1. So the table becomes as shown in below.

Process	Allocation		Request	
	Resource		Resource	
	R1	R2	R1	R2
P1	1	0	0	1
P2	0	1	1	0
P3	0	1	1	0

So, the Available resource is = (0, 0), but requirement are (0, 1), (1, 0) and (1, 0). So you can't fulfill any one requirement. Therefore, it is in deadlock. Therefore every cycle in a multi-instance resource type graph is not a deadlock. If there has to be a deadlock, there has to be a cycle. So

in case of RAG with multi-instance resource type, the cycle is a necessary condition for deadlock but not sufficient.

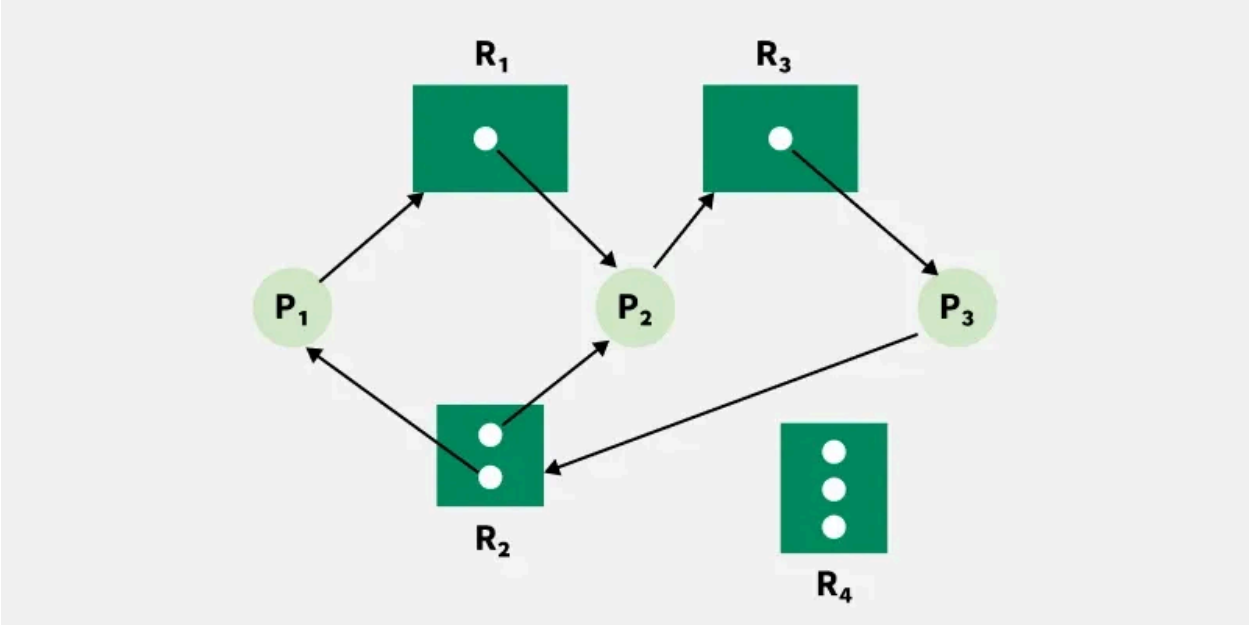
Deadlock Detection using Resource Allocation Graph

A Resource Allocation Graph (RAG) is used to detect deadlocks by analyzing the relationships between processes and resources in a system. It visually represents how resources are allocated and requested.

- If graph contains no cycles no deadlock.
- If graph contains a cycle if only one instance per resource type, then deadlock.
- If graph contains a cycle if several instances per resource type, possibility of deadlock

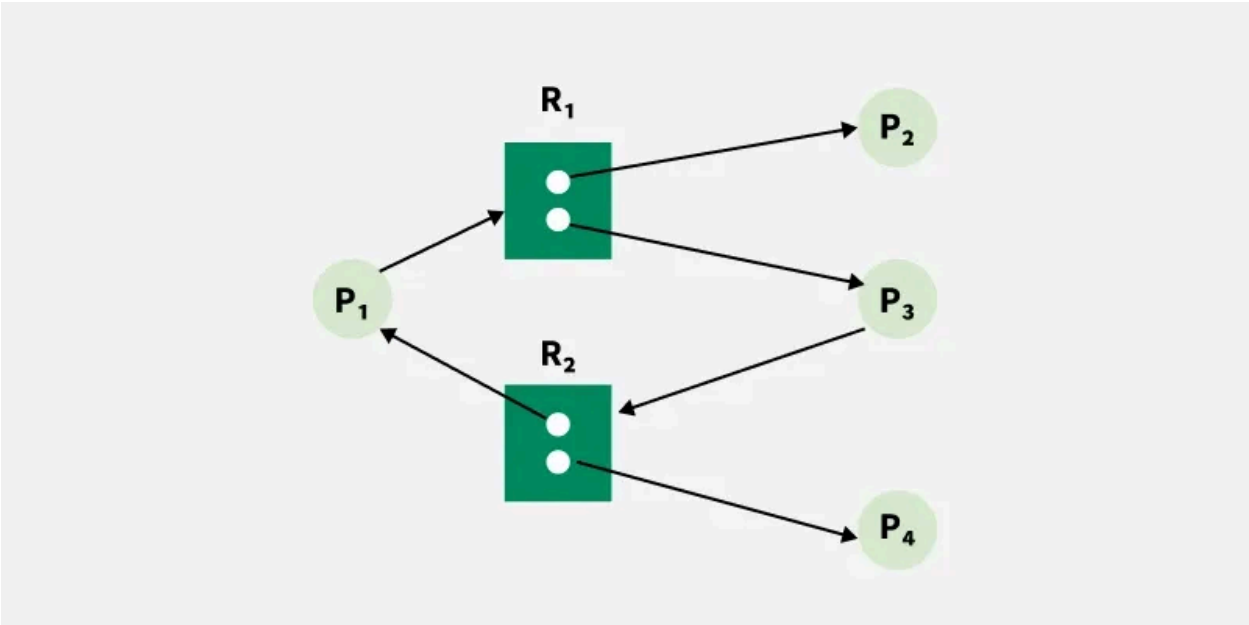
Figure given below is an example of System in a Deadlock because all processes are blocked and cannot proceed

- P1 is waiting for R1, held by P2.
- P2 is waiting for R3, held by P3.
- P3 is waiting for an instance of R2, whose one instance is held by p1 and other is held by p2l.



Following is an example of Resource Allocation Graph with a cycle but no Deadlock

- The system has enough resources to break the cycle.
- At least one process in the cycle can complete and release resources, allowing the remaining processes to proceed.


R1
P2
P1
P3
R2
P4